Qwen

Hexasphere Mathematics for Trixel: A Specification for Topologically-Aware Curvature Proofs

Core Construction Pipeline: From Seed Icosahedron to Dual Polyhedron

The generation of a hexasphere, a spherical mesh composed predominantly of hexagonal faces with exactly twelve pentagonal defects, follows a well-established, multi-stage mathematical pipeline [1] [8]. This process begins with a Platonic solid as a base shape and proceeds through subdivision and duality transformations to produce the final Goldberg polyhedral structure [31]. An analysis of open-source implementations reveals a consistent sequence of operations that can be abstracted into a language-agnostic specification. The first stage involves the construction of a seed polyhedron, overwhelmingly identified as a regular icosahedron due to its high symmetry and equilateral triangular faces, which provide an ideal basis for a uniform tessellation [1] [8] [31]. The second stage is the deterministic subdivision of each of the icosahedron's faces, controlled by a frequency parameter that dictates the level of detail [7] [24]. The third and most critical stage is the construction of the dual graph, which transforms the vertices and edges of the subdivided mesh into the faces and adjacency relationships of the final polygonal grid [29] [30]. This sequence—seed, subdivide, dualize—forms the foundational logic that must be precisely reverse-engineered and adapted.

The mathematical construction of the seed icosahedron is a `TRIXEL_NEW` component, representing a fundamental design choice for achieving spherical hexagonal tiling. Its vertices are not arbitrary but are defined using the golden ratio, $\varphi = (1 + \sqrt{5}) / 2$, to ensure all edges are of equal length and all faces are perfectly equilateral triangles [9]. The twelve vertices are given by all possible even permutations of the coordinates $(\pm 1, \pm \varphi, 0)$, $(0, \pm 1, \pm \varphi)$, and $(\pm \varphi, 0, \pm 1)$ [9]. This specific coordinate system results in a polyhedron with 12 vertices, 30 edges, and 20 triangular faces [1]. This inherent topology is non-negotiable; it is a direct consequence of Euler's formula ($V - E + F = 2$) and the geometric constraints of spherical topology, which dictate that a closed surface tiled with hexagons must contain exactly 12 pentagonal defects to accommodate the positive

curvature at those points [11] [27]. The choice of the icosahedron over other Platonic solids like the octahedron is primarily driven by its higher number of faces, which allows for a more gradual and less distorted tessellation when projected onto a sphere [10]. Therefore, while the concept of using a Platonic solid is general, the specific implementation of a regular icosahedron with golden-ratio-derived coordinates is a foundational element of the hexasphere algorithm.

Following the creation of the seed icosahedron, each of its 20 triangular faces undergoes a deterministic subdivision process. This operation increases the resolution of the mesh and creates the underlying triangular lattice upon which the final polygonal cells will be built. The literature describes several methods for this subdivision, all of which are `TRIXEL_ADAPTED`. One common approach is iterative edge splitting, where the midpoints of each triangle's edges are found and connected, thereby subdividing one large triangle into four smaller ones [9]. This method is simple and highly predictable. Another, more sophisticated technique is the Loop subdivision algorithm, which adds new vertices at the midpoints of edges but then repositions them to a weighted average of their neighboring vertices to create a smoother surface [31]. Some implementations use generalized interpolation along the edges of the original triangle, which can be linear or normalized, to generate the new points [8]. The frequency or "divisions" parameter controls the number of times this subdivision process is applied to the original icosahedron's faces [7] [24]. For instance, a frequency of `n` means each edge of the original icosahedron will pass through `n` centers of hexagonal tiles in the final mesh [1]. The choice of subdivision algorithm represents a trade-off between computational simplicity and surface smoothness. For the purpose of establishing a deterministic proof-of-concept within Trixel, a simpler, more transparent method like iterative edge splitting would be preferable, as it minimizes ambiguity in the resulting mesh structure.

The third and most crucial stage in the pipeline is the generation of the dual-cell structure. This transformation directly produces the final mesh of hexagons and pentagons and establishes the primary adjacency relationships. The dual of a polyhedron is formed by placing a new vertex at the center of each face of the original polyhedron and connecting these new vertices if their corresponding faces shared an edge in the original [11] [29]. In the context of the hexasphere, the "center" of a face can be calculated as its centroid—the average of its vertices' coordinates [9]. After constructing these new vertices (cell centers), polygons are formed by connecting a central vertex to the centers of its adjacent faces and the midpoints of the shared edges [9]. This duality operation is the mechanism that converts the problem of tiling a sphere with hexagons/pentagons into the problem of subdividing a triangular mesh [30]. The topological necessity of the 12 pentagons becomes immediately apparent at this stage: the 12 original vertices of the

icosahedron correspond to the 12 faces in the dual mesh. Since the degree of each of these original vertices is five (five edges meet at each icosahedron vertex), the dual operation produces 12 pentagonal cells [1] [9]. All other faces in the dual mesh, corresponding to the interior of the subdivided triangular faces, become hexagons. This process guarantees the correct topology without any ad-hoc adjustments. The resulting structure is known as a Goldberg polyhedron, which is formally defined as the dual of a geodesic sphere (the subdivided icosahedron) [10] [31]. This `construct_dual()` operation is the cornerstone of the entire algorithm, as it generates the complete topological graph of cells and their neighbors before any geometric embedding is considered. This ensures that the topology precedes the geometry, a core tenet of the Trixel doctrine.

The final step in defining the abstract mesh is the calculation of the individual polygon corners. These corner points are the vertices of the final Goldberg polyhedron. They are derived from the structure of the dual mesh. As described previously, the corners of a new polygon are often the midpoints of the edges from the original subdivided icosahedron, combined with the centroids of the faces [9]. A more advanced method for ensuring that the resulting faces are planar involves a more complex definition of the "center point" of a triangle on the sphere. Instead of a simple centroid, this center is defined as the intersection point of the three planes that are tangent to the sphere at each of the triangle's three corners [10] [33]. This method, while computationally more intensive, provides a mathematically rigorous way to construct a dual mesh with planar faces, which is a desirable property for many applications [33]. Once all the corner points for all polygons have been calculated in their canonical, pre-embedded state, the entire structure exists as a topological graph (the PolygonCellGraph) with a set of associated sample/corner points. At this stage, no part of the geometry has yet been projected onto a curved surface; it is a purely topological object. The subsequent section on geometric embedding details how this abstract structure is mapped to a sphere while attempting to preserve the intrinsic properties of its constituent cells.

Stage	Operation	Description	Trixel Classification
1	`build_seed_icosahedron()`	Constructs a regular icosahedron with 12 vertices, 30 edges, and 20 equilateral triangular faces, defined using the golden ratio for vertex coordinates [1] [9].	`TRIXEL_NEW`
2	`subdivide_faces(seed_topology, frequency)`	Iteratively splits each triangular face into smaller triangles. Methods include edge splitting [9] or Loop subdivision [31], controlled by a frequency parameter [7].	`TRIXEL_ADAPTED`
3	`construct_dual(subdivided_mesh)`	Creates a new mesh where each face of the input becomes a vertex. Adjacency is inherited from shared edges, producing a graph of hexagonal and 12 pentagonal cells [23] [29] [31].	`TRIXEL_ADAPTED`
4	`derive_ordered_polygon_corners(dual_graph)`	Calculates the corner points of each polygon. This can involve finding midpoints of original edges and centroids [9] or computing intersections of tangent planes for planar faces [10].	`TRIXEL_ADAPTED`

This systematic deconstruction of the construction pipeline provides a clear, logical framework for extracting the necessary mathematics. Each stage builds upon the previous one, culminating in a fully specified topological graph that embodies the desired hexasphere structure. The separation of these stages is critical, as it allows for independent validation and adaptation of each component, particularly the `construct_dual` operation, which is the linchpin holding the topological authority of the entire system.

Geometric Embedding and Curvature Quantification

Once the abstract topological structure of the hexasphere—a dual-graph spherical grid—is fully defined with its cell centers and corner points, the next stage is its geometric embedding onto a curved surface, typically a sphere [6]. This process, referred to as `embed_samples()`, is where the flat-topology mesh acquires curvature. However, this transformation inherently risks distorting the intrinsic properties of the mesh's cells, such as their edge lengths, areas, and shapes. The user's directive mandates a rigorous, quantitative approach to evaluating this process, moving beyond visual assessment to measure specific distortions. The core principle guiding this evaluation is Trixel's doctrine that embeddings should preserve the intrinsic structure of cells; any failure to do so indicates that the operation has crossed into the realm of topology modification, which

requires explicit authorization [20]. To achieve this, the research must employ a suite of quantitative metrics and subject the embedded geometry to a series of curvature tests (A-E). These tests will differentiate between desirable curvature, which arises from changing the angles between facets (dihedral angles), and undesirable distortion, which manifests as stretching, compression, or shearing of the faces themselves.

The primary methods for projecting the flat, subdivided mesh onto a sphere vary in complexity and their impact on metric preservation. The simplest and most commonly cited method is Gnomonic Projection, also known as normalization. In this technique, every vertex's Cartesian coordinate vector is divided by its magnitude, effectively scaling it to lie on the surface of a unit sphere centered at the origin [12]. While computationally inexpensive, this method is known to introduce significant area distortion, making tiles near the corners of the original icosahedral faces much larger than those near the centers [1] [12]. A more sophisticated approach is the use of Spherical Linear Interpolation (slerp). During the subdivision phase, instead of interpolating linearly between points, slerp is used to interpolate along the arc of a great circle on the sphere [8] [14]. This helps maintain a more uniform distribution of points and can lead to a more accurate final mesh, though it is more computationally expensive than simple linear interpolation [8]. For applications demanding high accuracy, specialized projections like Snyder's equal-area projection can be employed. These methods are designed to minimize area distortion at the cost of significantly increased computational complexity and implementation difficulty [1] [12]. The choice of projection method is a critical decision that directly influences the outcome of the curvature tests. For the initial Trixel proofs, a baseline comparison between a simple Gnomonic Projection and a more accurate method like slerp would be highly informative.

To objectively evaluate the quality of any embedding, a set of quantitative metrics must be tracked for each cell before and after the `embed_samples()` operation. These metrics form the basis for verifying whether the embedding preserves the intrinsic geometry of the cells. The user's directive specifies a clear hierarchy of measurements. First, and most critically, is the measurement of metric distortion:

- **Edge Length Change:** The difference between the original edge lengths in the canonical, flat-topology mesh and the embedded edge lengths on the sphere. A successful embedding will show minimal change, indicating that the faces were not stretched or compressed.
- **Area Change:** The ratio of the embedded face area to its canonical area. This measures overall scaling distortion. Equal-area projections are specifically designed to keep this value close to 1 [1].

- **Aspect Ratio Change:** The change in the ratio of a polygon's longest side to its shortest side. A value close to 1 indicates a well-proportioned, undistorted shape. Distortions tend to be most pronounced in hexagons surrounding the pentagonal defects [11].

Second, it is important to measure the introduction of curvature itself, which is distinct from distortion. These metrics confirm that the surface is indeed becoming curved:

- **Face Normal Deviation:** The angle between the normal vector of a face in its canonical state (which would be constant for a flat patch) and its normal vector after embedding. This deviation directly quantifies the amount of local curvature introduced at that face. This is an expected and desirable outcome of the embedding process.
- **Dihedral Angle Change:** The angle between the planes of two adjacent faces. In the canonical flat state, this angle is 180 degrees. After embedding, this angle will deviate from 180 degrees, representing the "bending" or folding of the surface. Like face normal deviation, this is an expected and desirable effect of curvature.

Finally, and above all else, is the verification of topological integrity. The adjacency graph must remain unchanged by the geometric embedding.

- **Adjacency Integrity:** The most stringent test is to compare the neighbor graph before and after embedding. According to the Trixel doctrine, `GRAPH_BEFORE_EMBEDDING` must be identical to `GRAPH_AFTER_EMBEDDING` for a valid curvature proof [19] [21]. Any change in declared adjacencies constitutes a catastrophic failure of the topological authority principle.

These metrics provide a comprehensive framework for analyzing the embedding process. The goal is to answer the fundamental question posed in the research directive: Did the surface acquire curvature primarily through changing the angles between facets (dihedral angles), or did it happen through distorting the faces themselves (stretching)? By rigorously measuring the metrics above, the team can generate objective, quantitative evidence. A successful embedding would show significant changes in face normal and dihedral angle, but minimal changes in edge length, area, and aspect ratio, with zero change in the adjacency graph.

The following table outlines the expected behavior of these metrics across a series of planned curvature tests, providing a clear benchmark for success.

Test Case	Description	Expected Face Normal Deviation	Expected Dihedral Angle Change	Expected Edge Length Change	Expected Area Change	Expected Adjacency Integrity
TEST A	A small, flat polygon patch.	Minimal (near-zero)	Minimal (near-zero)	Minimal (should be zero)	Minimal (should be zero)	Must Pass
TEST B	Same patch from TEST A bent by 45°.	Significant	Significant	Minimal	Minimal	Must Pass
TEST C	Same patch from TEST A bent by 90°.	Significant	Significant	Minimal	Minimal	Must Pass
TEST D	Same patch from TEST A formed into a cylinder.	Varies by location	Significant	Minimal (along length)	Minimal	Must Pass
TEST E	Compound curvature (e.g., saddle shape).	High	High	Potentially High	Potentially High	May Fail

In TESTS A-D, the objective is to prove that pure curvature can be achieved while preserving the intrinsic metrics of the cells. If the metrics show acceptable tolerance levels, it validates the core mathematical pipeline. TEST E, however, is the ultimate stress test. It is theorized that under extreme compound curvature, it may become mathematically impossible to preserve the intrinsic metrics of all cells simultaneously without introducing a topological change, such as modifying the hexagonal lattice by adding heptagons or removing hexagons to accommodate negative curvature [27]. A failure in adjacency integrity or a massive spike in metric distortion during TEST E would be a significant finding, potentially leading to the discovery of a `TRIXEL_NEW` topological modification rule that Trixel could adopt. Visual inspection remains a valuable secondary tool throughout these tests, providing qualitative confirmation of the quantitative data—for example, visually confirming that a region that shows low edge-length change has not been "smeared" or "squeezed" [25].

Auditing Adjacency Derivation: Validating Topological Authority

A central tenet of the Trixel research goal is the strict enforcement of topological authority, where the geometric embedding of a mesh must faithfully realize a pre-determined topological structure. This principle is encapsulated in the doctrine that compiled topology must equal embodied geometry, meaning that adjacency relationships cannot be invented or altered by post-processing based on geometric proximity [19] [21]. The user's directive places immense importance on auditing the open-source reference

libraries not just for their mathematical formulas, but for the very philosophy behind how they derive cell adjacency. The analysis of the provided sources reveals two fundamentally different and incompatible approaches to this problem: a topology-first methodology, which is valid and aligns with Trixel's doctrine, and a geometry-first methodology, which is invalid and must be rejected. A meticulous audit of each library against this distinction is therefore a prerequisite to safely adapting its mathematics.

The valid, topology-first approach derives adjacency from the inherent structure of the mesh at various stages of its construction. This method ensures that the neighbor graph is an integral and immutable part of the mesh's identity from its inception. Several examples of this methodology appear across the references. The most direct and robust method is derived from the dual-cell construction process itself. When generating the dual of the subdivided icosahedron, adjacency is defined by the incidence relationships of the original mesh. Two cell centers in the dual are connected by an edge if and only if their corresponding faces in the original subdivided mesh shared an edge [11] [29]. This process explicitly constructs the adjacency map as a byproduct of the dualization, guaranteeing that the graph is correct by definition. Another topology-first technique involves using half-edge data structures, which store explicit connectivity information, allowing for efficient traversal and determination of adjacent faces [22]. Furthermore, some Rust-based libraries are designed to generate adjacency maps directly from the indices produced during the subdivision process, implying a deterministic, index-based derivation of topology rather than a spatial search [2] [3] [4]. This category of methods respects the principle that topology generates geometry, not the other way around. The neighbor relationship is declared and recorded long before any vertex coordinates are finalized.

In stark contrast, the invalid, geometry-first approach infers adjacency from the final, spatial arrangement of the mesh. This method treats the generated mesh as a collection of points in space and attempts to deduce topological connections based on geometric proximity. This approach is fundamentally flawed from a topological perspective because it violates the principle that topology is about connectivity, not position. Several problematic patterns emerge from this methodology. One common pattern is the use of nearest-point searches, where for a given cell, its neighbors are identified as the cells whose centers are closest in XYZ coordinates [21]. This is susceptible to errors caused by mesh deformation or uneven point distribution. Another variant involves building a spatial index, such as a hash grid or tree, and then for each point, searching within its local bucket to find nearby points, declaring them to be neighbors [5]. This "closest means connected" logic is a clear violation of topological authority. A third, related method involves welding vertices that are closer than a certain threshold distance and then

traversing the resulting mesh to find shared edges [11]. While this can produce a valid mesh, the adjacency relationships are discovered after the geometry is already in place, making them dependent on the specific projection and potential numerical precision issues. These geometry-first methods are antithetical to the Trixel doctrine because they allow geometric proximity to override or invent topological reality.

The following table provides a comparative analysis of the two approaches, highlighting why the topology-first method is essential for the research goal.

Feature	Topology-First (Valid)	Geometry-First (Invalid)
Source of Adjacency	Derived from mesh structure (e.g., dual construction, incidence, half-edges) [22] [29]	Derived from spatial proximity (e.g., nearest-point search, distance thresholds) [5] [21]
Order of Operations	Topology is defined first; geometry is embedded later [9]	Geometry is generated first; topology is inferred afterward [11]
Determinism	Highly deterministic and repeatable, assuming identical inputs [8]	Can be sensitive to floating-point precision and implementation details of the proximity search [11]
Topological Integrity	Guarantees `GRAPH_BEFORE_EMBEDDING == GRAPH_AFTER_EMBEDDING` [19]	Cannot guarantee this equality; adjacency may be altered or incorrectly assigned during the inference step [21]
Trixel Doctrine Alignment	Fully compliant; topology precedes and dictates geometry	Violates doctrine; geometry can redefine topology

Auditing the specified open-source libraries against this framework is a critical task. The `AllphinsPilot/hexasphere` Python library appears promising as its documentation emphasizes the logical flow from subdivided triangles to tile centers and then to pentagons and hexagons, suggesting a strong topology-first foundation [9]. The `fwdssh/godot-hexasphere-generator` exposes a neighbor dictionary as part of its public API, but the source code would need to be examined to determine if this map is constructed from indices during generation or computed via a spatial query after the mesh is built [5]. The `OptimisticPeach/hexasphere` Rust library explicitly mentions features for generating adjacency maps from indices, which is a strong indicator of a topology-first approach [2] [4]. Any reference implementation that relies on a post-generation nearest-neighbor search or a similar geometry-first technique must be flagged as incompatible and its adjacency-related mathematics discarded. The goal is to salvage the proven, deterministic pipeline that generates topology from a structured process, not the heuristic, geometry-dependent methods that attempt to recover topology from a finished product. This rigorous auditing ensures that the final Trixel-compliant pipeline is built on a sound theoretical foundation where topology remains sovereign over geometry.

A Trixel-Compliant Specification and Pseudocode Framework

To translate the findings of this research into a durable and adaptable asset for the Trixel project, it is imperative to document the extracted mathematics in a formal, unambiguous manner. The user's directive specifies a two-layer documentation standard that serves both as an authoritative, language-agnostic mathematical specification and as a practical guide for implementation within Trixel's existing ecosystem. This dual-layer approach provides a crucial buffer, allowing the underlying mathematics to be replaced or refined in the future without disrupting the high-level logic of the Trixel proof scripts. Furthermore, a strict tagging convention—`REFERENCE_DERIVED`, `TRIXEL_ADAPTED`, and `TRIXEL_NEW`—must be enforced to maintain full traceability of the mathematical components. This ensures that future developers can understand the provenance of each piece of the system, distinguishing between established, external knowledge and novel contributions made by the Trixel team.

The first layer of documentation is the language-agnostic mathematical specification. This serves as the definitive blueprint for the hexasphere construction pipeline. It must describe the entire process in terms of inputs, operations, and outputs, with absolute clarity regarding ordering, determinism, and numeric assumptions. This specification acts as the contract that any implementation must adhere to. It should be structured around the core operations identified in the research, defining precisely what each function does.

Language-Agnostic Mathematical Specification

- **INPUTS:**
- `seed_topology`: A specification of the initial polyhedron (e.g., 'icosahedron').
- `subdivision_frequency`: An integer $\geqslant 1$, specifying the number of times each face is subdivided.
- `embedding_parameters`: A set of parameters controlling the geometric projection (e.g., projection_type='gnomonic', radius=1.0).

- **OPERATIONS:**
- `build_seed_icosahedron() -> SeedTopology`: Constructs a regular icosahedron. Vertex coordinates are calculated using the golden ratio, $\varphi = (1 + \sqrt{5}) / 2$. The output is a topological object containing 12 vertices, 30 edges, and 20 triangular faces, with all edges being of equal length [1] [9].
- `subdivide_faces(topology, frequency) -> SubdividedTopology`: Performs a deterministic subdivision on each triangular face. For a given frequency, it

iteratively connects edge midpoints to split each triangle into four smaller triangles [9]. The output is a new topological object with a finer triangular mesh.

- `construct_dual(topology) -> PolygonCellGraph`: Generates the dual graph of the input topology. For every face in the input, a new vertex (cell center) is created at its centroid [9][29]. An edge is added between two cell centers in the output if their corresponding faces in the input shared an edge. The output is a graph where nodes represent cells (12 pentagons and the rest hexagons) and edges represent adjacency [1].
- `derive_ordered_polygon_corners(dual_graph) -> CornerPointSet`: Computes the corner points for each polygon in the dual graph. For each cell, its corners are determined by the locations of the cell centers of its neighbors and the midpoints of the shared edges in the original mesh [9]. This operation produces a canonical, pre-embedded coordinate for every corner point.
- `embed_samples(graph, parameters) -> CanonicalGeometry`: Maps the abstract CornerPointSet onto a curved surface. For each corner point, its coordinates are transformed according to the specified projection method (e.g., Gnomonic projection involves normalizing the vector) [12]. The output is a concrete set of 3D coordinates realizing the topology in space.

- **OUTPUTS:**
- `cell_ids`: A unique identifier for each polygonal cell.
- `corner_sample_ids`: A unique identifier for each corner point.
- `polygon_incidence`: A mapping of each cell ID to the ordered list of its corner sample IDs.
- `neighbor_graph`: An adjacency list or matrix representing the compiled cell graph.
- `canonical_coordinates`: The final 3D coordinates of all corner samples.

The second layer of documentation is the immediate application of this specification within Trixel's own syntax. This Trixel-style pseudocode serves as the bridge to actual implementation and as the basis for the deterministic acceptance tests. It mirrors the patterns used in Trixel's existing proof scripts, ensuring consistency and ease of integration. Every formula or algorithm taken from a reference library must be tagged accordingly.

Trixel-Style Pseudocode

```
// Trixel-Compliant Hexasphere Generation Pipeline
// All harvested formulas are tagged with their origin.

// Step 1: Build the foundational topology
```

```
SeedTopology seed = build_seed_icosahedron();
// REFERENCE_DERIVED: Vertex coordinates from Wikipedia/Three.js using gol

// Step 2: Increase resolution deterministically
SubdividedTopology subdivided = subdivide_faces(seed, SubdivisionFrequency
// TRIXEL_ADAPTED: Algorithm based on iterative edge splitting <sup><a cla

// Step 3: Generate the final cell structure and adjacency
PolygonCellGraph cell_graph = construct_dual(subdivided);
// REFERENCE_DERIVED: Dual construction based on face adjacency <sup><a cl

// Step 4: Calculate the vertices of each cell
CornerPointSet corner_set = derive_ordered_polygon_corners(cell_graph);
// TRIXEL_ADAPTED: Corners are midpoints and centroids <sup><a class="ref-

// Step 5: Embodied the abstract structure onto a sphere
CanonicalGeometry geometry = embed_samples(corner_set, EmbeddingParameters
// REFERENCE_DERIVED: Projection method from library docs <sup><a class="r
// TRIXEL_ADAPTED: Wrapper around the chosen projection algorithm

// Verification Phase: Check adherence to Trixel's doctrine
if (verify_topology_preservation(cell_graph, geometry)) {
    // Accept the embedding as a valid proof of curvature
} else {
    // Reject the embedding; topology was violated by geometry
}

// Verification Function Example
function verify_topology_preservation(PolygonCellGraph pre_embed, Canonica
    // Compare the adjacency lists of the two graphs
    // This is a brutal check: the topology must be identical before and a
    return (pre_embed.get_adjacency_list() == post_embed.get_adjacency_lis
}
```

This two-layer documentation strategy provides maximum flexibility and safety. The mathematical specification is the permanent, authoritative record of the system's logic. The pseudocode is a transient, executable artifact for the current research and development cycle. Should a better algorithm for subdivision or projection be discovered in the future, only the `TRIXEL_ADAPTED` or `REFERENCE_DERIVED` functions would need to be rewritten, leaving the overarching `TRIXEL_NEW` logic and the high-level

structure intact. The strict tagging system prevents intellectual debt from accumulating, ensuring that the origins of all mathematical concepts are always clear. This disciplined approach directly supports the research goal by creating a robust, verifiable, and maintainable mathematical foundation for Trixel's curvature proofs.

Research Execution Plan: Isolated Proofs and Validation Metrics

To fulfill the research goal of developing a Trixel-compliant curvature embedding without prematurely integrating complex dependencies, a structured and isolated execution plan is essential. The user's directive provides a clear path forward: establish a dedicated research environment, dissect the selected open-source libraries in a specific order, and focus exclusively on extracting the core mathematical machinery. The output of this research phase should be a minimal, self-contained script that implements the validated mathematical pipeline and runs a predefined set of curvature tests. This proof-of-concept will serve as the definitive test of the extracted mathematics, providing quantitative evidence to support or refute the hypothesis that hexasphere geometry can achieve curvature while preserving intrinsic cell metrics. This section outlines the specific steps for executing this research, including the composition of the isolated directory, the prioritized study order of the references, and the explicit list of mathematical concepts to be extracted.

The first and most critical action is to establish an isolated research directory to contain all experimental code and analysis. Following the user's instruction, this directory should be located at `research/polygon_curvature/`. This isolation is paramount to prevent the accidental contamination of the main Trixel repository with incomplete, improperly vetted, or dependency-heavy code. The contents of this directory should be limited to the pure mathematics and the minimalist proof-of-concept script. It must explicitly exclude any and all elements from the reference libraries that are not directly related to the core mathematical pipeline. This includes renderer-specific code (e.g., Godot's ArrayMesh or Unreal Engine's rendering components [5] [15]), gameplay systems (e.g., planet generation logic, biome assignment, hydraulic erosion [6] [18]), shaders, input handling, and any custom data structures or utility classes that are not part of the core algorithm. The sole purpose of this directory is to house the mathematical derivation and its direct validation.

The study of the open-source references should proceed in a deliberate, hierarchical order to maximize efficiency and understanding.

1. **Primary Reference: `AllphinsPilot/hexasphere` (Python)**. This library is the ideal starting point for reverse-engineering the mathematics. Its pure Python implementation makes the core logic far easier to read and isolate compared to C++ or Rust projects embedded within larger engine ecosystems [5]. The available documentation explicitly breaks down the construction into a clear sequence: icosahedron → subdivision → tile centers → pentagons/hexagons → projection [9]. The researcher should begin here to develop a working understanding of the entire pipeline and to draft the initial language-agnostic specification and pseudocode. The goal is to replicate the mathematical steps in a clean, Trixel-native environment.

2. **Secondary Reference: `fwdssh/godot-hexasphere-generator` (C++ GDExtension)**. Once a basic implementation exists based on the Python reference, this library should be studied to see how the same mathematical concepts translate into a real-world engine context. Its strength lies in its documented API, which exposes key outputs like tile centers, corner points, and a neighbor dictionary [5]. The focus of this analysis should be twofold: first, to verify that the outputs match the expectations set by the mathematical model, and second, to perform the critical audit of its adjacency derivation. The documentation suggests the neighbor relationships are returned in a Compressed Sparse Row (CSR) format, which implies a pre-computed, topology-based structure rather than a runtime proximity search [5]. This makes it a valuable source for validating the `construct_dual` and `embed_samples` stages.

3. **Tertiary Reference: `OptimisticPeach/hexasphere` (Rust)**. This library serves as a final point of comparison and validation. Its documentation highlights features related to adjacency maps and spherical interpolation (slerp), making it a useful resource for exploring alternative implementations or optimization techniques [4] [8]. Studying this crate can help identify more robust or accurate methods for projection that could be adapted into the Trixel pipeline, potentially improving the results of the curvature tests.

From these three libraries, the researcher must extract and document only a narrow set of mathematical concepts, as specified in the initial conversation. These five items form the core of the research deliverable.

Extracted Component	Description	Purpose in Trixel
Seed Polyhedron Construction	The mathematical definition of the initial icosahedron, including vertex coordinates derived from the golden ratio. [1] [9]	Provides the foundational, symmetric topology required for a hexagon/pentagon mesh.
Subdivision Algorithm	The deterministic process for refining the triangular faces of the seed polyhedron. [9] [31]	Controls the resolution and density of the final mesh.
Dual-Cell Construction	The method for generating the final polygonal cells (mostly hexagons) and their inherent adjacency graph by taking the dual of the subdivided mesh. [23] [29]	Establishes the topological authority of the cell graph before any geometry is considered.
Cell Corner Generation	The algorithm for calculating the precise corner points of each polygon in its canonical, pre-embedded state. [9] [10]	Defines the intrinsic shape of each cell prior to curvature.
Projection/ Embedding Mathematics	The formulas and methods for mapping the abstract, flat-topology mesh onto a curved surface (e.g., a sphere). [8] [12]	Transforms the proof-of-concept from a flat plane into a curved space for testing.

By adhering to this structured plan—creating an isolated environment, studying the references in order, and focusing solely on these five mathematical components—the research can systematically harvest proven mathematics while avoiding the common pitfall of accidentally importing unwanted architectural assumptions or dependencies. The final output will be a lean, focused, and mathematically sound script that serves as the bedrock for all subsequent work on Trixel's curvature and topological doctrines.

Synthesis and Strategic Recommendations

This deep research report has systematically deconstructed the mathematical principles behind hexasphere construction by auditing open-source reference implementations against the stringent requirements of the Trixel project. The analysis confirms that a robust, language-agnostic pipeline for generating a topologically-aware, curvature-preserving mesh exists and can be reverse-engineered. The core of this pipeline consists of four sequential stages: the construction of a seed icosahedron, the deterministic subdivision of its faces, the generation of a dual-cell graph to establish topology, and the geometric embedding of this topology onto a curved surface. The user's directive to prioritize topological authority—ensuring that compiled topology equals embodied geometry—is not only feasible but is the central organizing principle that distinguishes a valid implementation from an invalid one. The strategic recommendations derived from this analysis provide a clear and actionable path forward for the research and development team.

The primary strategic recommendation is to proceed with the extraction and validation of the hexasphere mathematics as outlined in this report, but to execute it within the prescribed isolated research environment (`research/polygon_curvature/`). This approach mitigates risk by preventing premature architectural commitments and ensuring that only vetted mathematical concepts are integrated. The research must prioritize the auditing of adjacency derivation methods, explicitly rejecting any implementation that relies on post-hoc geometric proximity checks in favor of those rooted in topological incidence, such as dual construction or index-based mapping [2] [29]. The `construct_dual` operation is the keystone of this effort; its successful implementation is the single most important factor in upholding the Trixel doctrine that topology must precede geometry.

The second major recommendation is to embrace the two-layer documentation framework: a language-agnostic mathematical specification followed immediately by Trixel-style pseudocode. This dual-layer system is not merely a best practice but a critical safeguard for the project's long-term maintainability. The specification serves as the immutable, authoritative source of truth for the algorithm's logic, while the pseudocode provides a direct, executable link to the existing Trixel proof infrastructure. The strict enforcement of the `REFERENCE_DERIVED`, `TRIXEL_ADAPTED`, and `TRIXEL_NEW` tagging convention will be instrumental in managing technical debt and ensuring that the lineage of every mathematical component is transparent and auditable. This discipline will prevent the accumulation of "black box" code and empower future developers to reason about and modify the system with confidence.

The third recommendation is to treat the proposed curvature tests (A-E) not as optional exercises but as the primary validation criteria for the entire research effort. The quantitative metrics—edge length change, area change, aspect ratio change, and, most importantly, adjacency integrity—provide the objective, empirical evidence needed to answer the project's core questions. Success in Tests A-D, demonstrating that a valid curvature embedding can be achieved while preserving intrinsic cell metrics, would constitute a significant victory for the Trixel doctrine. Failure in these tests would pinpoint the exact stage of the pipeline (e.g., a particular projection method or dual-cell corner calculation) where distortion becomes unavoidable, providing invaluable diagnostic information. Test E, the compound curvature test, is the ultimate arbiter; a failure here would signal that Trixel's current topological model may need to be extended to include topological defects like heptagons to handle all possible curvatures, representing a profound but manageable evolution of the project's core principles.

In summary, the research confirms that the necessary mathematics for a Trixel-compliant curvature embedding are not a myth to be invented from scratch but a set of proven

techniques waiting to be responsibly adopted and adapted. The path forward is clear: meticulously extract the topology-first logic from the open-source references, discard the geometry-first approaches, and build a minimalist proof-of-concept that rigorously validates the extracted mathematics against a strict quantitative framework. By doing so, the Trixel project can move forward with a solid, provably correct mathematical foundation for its geometry pipeline, resolving the challenge of curvature while steadfastly upholding the primacy of topological authority.

Reference

1. AllphinsPilot/hexasphere: A module to create hexagonal ... https://github.com/AllphinsPilot/hexasphere
2. hexasphere - crates.io: Rust Package Registry https://crates.io/crates/hexasphere
3. OptimisticPeach/rehexed: Adjacency lists for hexasphere's ... https://github.com/OptimisticPeach/rehexed
4. hexasphere - Rust https://docs.rs/hexasphere
5. fwdssh/godot-hexasphere-generator https://github.com/fwdssh/godot-hexasphere-generator
6. Hexasphere Generator - Godot Asset Library https://godotengine.org/asset-library/asset/5195
7. Basic hexasphere implementation for Unity https://github.com/Em3rgencyLT/Hexasphere
8. OptimisticPeach/hexasphere: Subdividing triangle meshes. https://github.com/OptimisticPeach/hexasphere
9. Generating icosahedrons and hexspheres in Rust https://www.hallada.net/2020/02/01/generating-icosahedrons-and-hexspheres-in-rust.html
10. Split triangle sphere grid to hexagonal (Goldberg ... https://stackoverflow.com/questions/70143993/split-triangle-sphere-grid-to-hexagonal-goldberg-hexasphere
11. Hex Planets Dev Blog I: Generating the Hex Sphere https://devforum.roblox.com/t/hex-planets-dev-blog-i-generating-the-hex-sphere/769805
12. Generate Geodesic Sphere from Hex and Pentagons https://gamedev.stackexchange.com/questions/112702/generate-geodesic-sphere-from-hex-and-pentagons

13. IcoSphere in hexasphere::shapes - Rust https://doc.qu1x.dev/bevy_trackball/hexasphere/shapes/type.IcoSphere.html
14. Hexglobe https://lib.rs/crates/hexglobe
15. HexaSphere Library https://pandoa.github.io/HexaSphere/
16. Sphere hexagon subdivision - Grasshopper - McNeel Forum https://discourse.mcneel.com/t/sphere-hexagon-subdivision/83901
17. WebGL2 : 084 : Hex Sphere https://www.youtube.com/watch?v=PrCuCAJLRBo
18. Hexagonal tiling of the two-dimensional sphere https://pub.ista.ac.at/~edels/hexasphere/
19. A Topological-Geometrical Pipeline for 3D Cracking-like ... https://www.scitepress.org/Papers/2018/65527/65527.pdf
20. TriFlow: Generating Artist-Like 3D Mesh Topology via ... https://arxiv.org/html/2606.20131v1
21. 6.5 Topology and Relationships Between Geometric Primitives https://courses.ems.psu.edu/geog160/node/1949
22. Efficient traversal of mesh edges using adjacency primitives https://hhoppe.com/edgetrav.pdf
23. hexasphere - crates.io: Rust Package Registry https://crates.io/crates/hexasphere/19.0.0
24. Hexasphere/README.MD at master https://github.com/Em3rgencyLT/Hexasphere/blob/master/README.MD
25. Mathematically producing sphere-shaped hexagonal grid https://stackoverflow.com/questions/46777626/mathematically-producing-sphere-shaped-hexagonal-grid
26. Dual hexagon (rotational fractal) tessellation. https://www.facebook.com/groups/tiling/posts/1815314928892044/
27. Hexagon Tessellation Freeform Surface with constant angles? https://discourse.mcneel.com/t/hexagon-tessellation-freeform-surface-with-constant-angles/122170
28. (PDF) Equal Edge Hexagonal Spherical Tessellations https://www.researchgate.net/publication/281066653_Equal_Edge_Hexagonal_Spherical_Tessellations
29. Dual Tessellation -- from Wolfram MathWorld https://mathworld.wolfram.com/DualTessellation.html
30. An Introduction to Hexagonal Geometry https://ptacts.uspto.gov/ptacts/public-informations/petitions/1540062/download-documents?artifactId=ltCwcpg2j25vJ-4Yj9xattDu07y89Ke87MDdoW71wcXBPYTLp71g5jY

31. Generating Meshes of a Sphere https://danielsieger.com/blog/2021/03/27/generating-spheres.html
32. 3d - How to orient a hexagonal tile on a geodesic sphere ... https://gamedev.stackexchange.com/questions/125201/how-to-orient-a-hexagonal-tile-on-a-geodesic-sphere-goldberg-polyhedron
33. Hexagonal Meshes with Planar Faces https://www.cs.hku.hk/data/techreps/document/TR-2008-13.pdf